Reliable Job Queue with Dead Letter Recovery

Interview study guide generated from the actual TypeScript, Node.js, Redis, PostgreSQL, and Docker codebase.

Read this as your defense script: state what exists, explain why it exists, name the trade-offs, then say what you would add for production.

Source anchors: Repository inspected on 2026-08-07. Key files: README.md; package.json; Dockerfile; docker-compose.yml; packages/core/src/*; packages/api/src/*; packages/worker/src/*; packages/dashboard/src/*; packages/loadtest/src/*.

What you built in one sentence

- A distributed background job system where HTTP clients enqueue work, Redis provides the fast ready queue and per-job locks, PostgreSQL stores the durable job ledger, workers process jobs concurrently, failed jobs retry with exponential backoff, and exhausted jobs move to a replayable dead letter queue.

- The intended processing guarantee is at-least-once, not exactly-once. Idempotency keys dedupe enqueue requests for 24 hours; downstream job handlers still need idempotent side effects.

What to say if the interviewer asks for the punchline

- Redis is used for speed and atomic claim operations. PostgreSQL is used for durability, queryability, DLQ state, audit history, and worker heartbeats.

- Lua is used where multiple Redis commands must behave like one critical section: find a ready job, choose highest priority in a batch, acquire a lock, and remove it from the queue.

- The main honest limitation is cross-store consistency. Redis and PostgreSQL are coordinated in application code, not in a distributed transaction. A production version needs reconciliation or an outbox pattern.

1. Project Overview

Plain language

- Some work should not happen inside a user-facing request. Sending email, calling webhooks, processing payments, or syncing data may be slow or flaky.

- This project lets an API accept work quickly, store it safely, and let worker processes do the work in the background.

- If a job fails, it is retried later. If it keeps failing, it is preserved in a dead letter queue so an operator can inspect and replay it.

Technical language

- The API exposes `POST /jobs`, `GET /jobs`, `GET /jobs/:id`, `GET /dlq`, and `POST /dlq/:id/replay` routes.

- The queue engine stores canonical job state in `jobs` and `dead_letters`, while Redis sorted sets hold claimable job IDs scored by `available_at` epoch milliseconds.

- Workers poll Redis, atomically claim jobs through Lua, mark jobs processing in PostgreSQL, run a handler, then ack or nack.

Source anchors: packages/api/src/routes/jobs.ts:13,42,55; packages/api/src/routes/dlq.ts:13,26; packages/core/src/queue.ts:28,104,176,287; packages/core/src/schema.sql:7,53.

Who would use it

- Backend services that need async, retryable work without blocking API latency.

- Operations teams that need to see queue depth, retry rate, worker liveness, and DLQ entries.

- Developers who need local Docker Compose infrastructure for demos, load tests, and interview-level system design discussion.

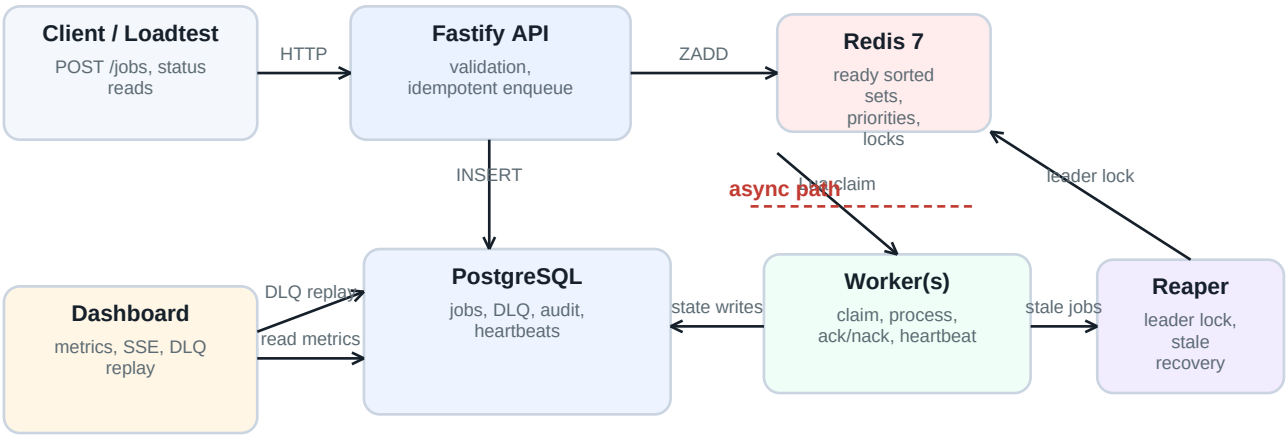
What done looks like

- A client enqueues a job and receives a job record immediately.

- A worker claims it once at a time, processes it, and writes final state to PostgreSQL.

- Successful jobs become completed with optional result JSON. Failing jobs become pending again with a delayed availability time until max attempts are exhausted. Exhausted jobs become dead and get a DLQ record.

2. System Architecture

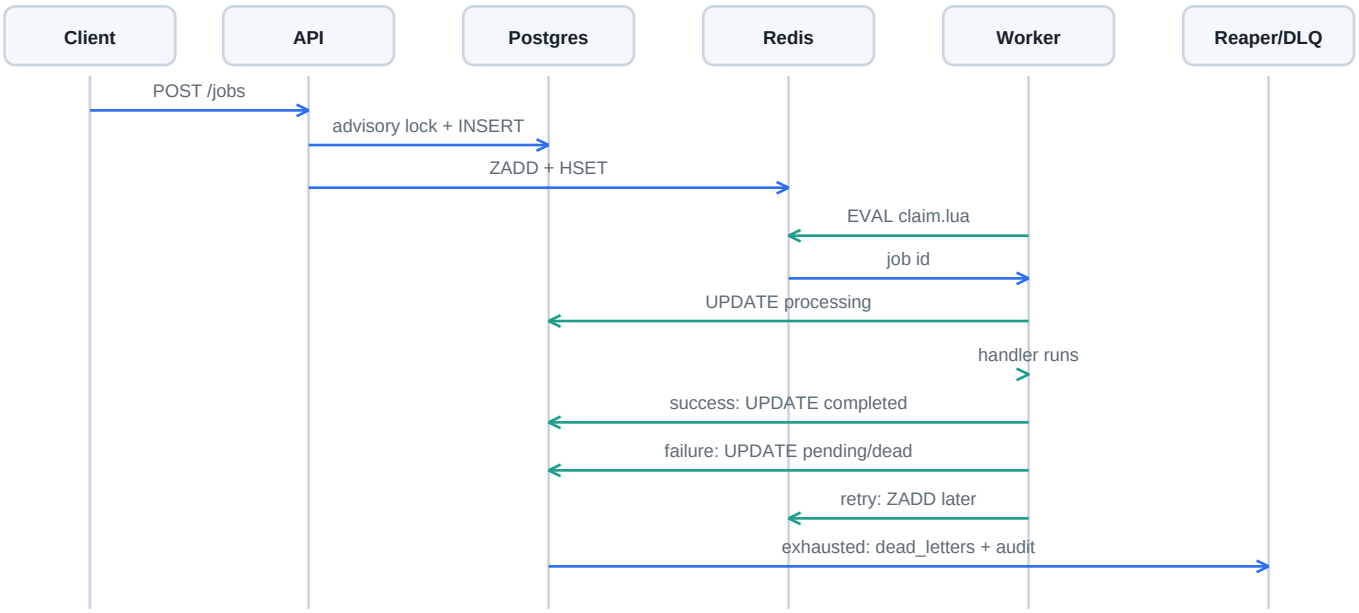


Source anchors: README.md architecture summary; docker-compose.yml:34-87; packages/core/src/redis.ts:52-67; packages/core/src/schema.sql:7-84.

Implemented components

Component	Current implementation	Interview defense
API	Fastify service on port 3000. Runs migration at startup, registers job and DLQ routes.	Small, typed HTTP boundary. API stays thin and delegates queue semantics to core.
Core queue service	Shared TypeScript package used by API, worker, dashboard, and loadtest.	Keeps Redis/Postgres coordination in one place instead of duplicating queue rules.
Redis	Sorted set per queue, priority hash per queue, per-job lock keys, reaper leader key.	Fast atomic operations for the hot path; Lua protects claim races.
PostgreSQL	jobs, dead_letters, worker_heartbeats, audit_log.	Durable source of truth and easy operational queries.
Workers	Poll loop plus semaphore; default concurrency 5; two workers intended in Compose.	Horizontal worker scaling is just more consumers sharing Redis and Postgres.
Dashboard	Fastify static HTML, metrics endpoint, SSE stream, DLQ replay routes.	Operational surface for monitoring and recovery.

Core Job Lifecycle



Visibility timeout is the Redis lock TTL plus jobs.locked_until; expired processing rows are requeued by the leader reaper.

Lifecycle notes

- Enqueue: if an idempotency key exists, PostgreSQL advisory transaction lock serializes same key requests before lookup and insert.

- Ready scheduling: Redis sorted set score is the job's available_at timestamp. Delayed jobs simply have a future score.

- Claim: Lua reads up to 50 ready candidates, picks the highest numeric priority, obtains a Redis lock with PX TTL and NX, removes the job from the ready set, then TypeScript marks it processing in PostgreSQL.

- Success: ack updates PostgreSQL to completed, clears lock fields, deletes the Redis lock, and removes priority hash metadata.

- Failure: nack either reschedules with exponential backoff or moves the job to dead_letters and audit_log.

- Recovery: the leader reaper scans PostgreSQL for processing jobs whose locked_until is expired and re-adds them to Redis.

Source anchors: packages/core/src/queue.ts:46-90,112-137,145-169,176-220,228-282,351-386; packages/core/src/lua/claim.lua:47-99.

Key Design Decisions

Decision	What exists in code	Why this design	Trade-off / limitation
Visibility timeout	Claim creates Redis lock with PX TTL and sets jobs.locked_until to now + visibilityTimeoutMs. Default 30 seconds.	Protects against worker crashes by letting expired work be reclaimed.	No lock extension heartbeat. Long-running handlers can be requeued and processed twice.
Lua atomic claim	claim.lua does ZRANGEBYSCORE, priority selection, SET NX PX, then ZREM as one Redis script.	Avoids two workers seeing or claiming the same ready job between separate Redis calls.	Atomic only inside Redis. The following PostgreSQL update is a second-store step.
Priority model	Priority stored in queue:<name>:priorities hash; Lua samples up to 50 ready jobs and chooses highest priority.	Keeps ready scheduling by time while still favoring important jobs.	Not a perfect global priority ordering if more than 50 jobs are ready.
Backoff formula	delay = min(baseMs * 2^attempt, capMs) + random jitter up to 25 percent. Defaults: base 1000 ms, cap 60000 ms.	Reduces immediate retry storms and spreads failures over time.	Jitter is random and hard to assert directly; tests cover deterministic base function.
Idempotency key	PostgreSQL advisory transaction lock on hash(queue_name:idempotency_key), then 24-hour lookup before insert.	Prevents concurrent duplicate enqueue requests from both passing the SELECT check.	Dedupe is enqueue-only; does not make job handler side effects exactly once.
DLQ replay	Replay resets original job to pending, attempts 0, clears error/locks, deletes DLQ entry, writes audit, and re-adds to Redis.	Keeps the original job id and lets operators retry after fixing the root cause.	Replay does not clone a new job; history is partially preserved in audit/DLQ, then DLQ row is deleted.

Source anchors: packages/core/src/config.ts:61-68; packages/core/src/backoff.ts:5-31; packages/core/src/queue.ts:46-58,192-213,287-342; packages/core/src/lua/claim.lua:47-99

Important honest defense: You chose an at-least-once design. That is normal for background queues. The right interview language is: Redis locks reduce duplicate claims, but the system still requires idempotent handlers because crashes, expired visibility timeouts, and cross-store failures can cause re-delivery.

3. Technology Stack Justification

Technology	Why used here	What could have been used	Trade-offs accepted
Redis	Low-latency sorted sets, hashes, TTL locks, and Lua atomicity for the hot claim path.	RabbitMQ, Kafka, AWS SQS, BullMQ, Postgres-only queue.	Fast but memory-bound. Single-threaded command execution per shard. Needs persistence/failover config in production.
PostgreSQL	Durable job source of truth, JSONB payloads/results, indexes, advisory locks, audits, heartbeats, operational queries.	MongoDB, DynamoDB, MySQL, Redis-only persistence.	Strong consistency and SQL at the cost of write throughput and connection management.
Node.js / TypeScript	Fastify API, async worker loop, shared typed core package, easy JSON payload handling.	Go, Java/Kotlin, Rust, Python.	Great I/O concurrency; CPU-heavy jobs would need separate processes or another runtime.
Docker	Compose brings up Postgres, Redis, API, worker, dashboard with health dependencies. Dockerfile builds package-specific runtime image.	Native local services, Kubernetes manifests, Terraform-only cloud setup.	Great reproducibility; Compose is not the production scheduler.
Lua scripts	Atomic multi-command Redis claim and reaper lock renewal/release compare-and-set.	WATCH/MULTI/EXEC, Redlock library, database row locks.	Small scripts are powerful, but must be kept simple and tested.
Fastify	Lightweight HTTP server with route modules and logging.	Express, NestJS, Hapi.	Less framework structure than NestJS; validation is currently minimal.

Source anchors: package.json:5-18; packages/*/package.json; Dockerfile:6-59; docker-compose.yml:1-90; packages/core/src/redis.ts:52-67.

Redis vs RabbitMQ vs SQS

- Redis fits a custom learning/project queue because sorted sets, TTL locks, and Lua let you implement delay, priority, and visibility mechanics directly.

- RabbitMQ gives mature broker semantics, routing, acknowledgements, and durability but less direct control over custom idempotency and Postgres-backed job state.

- SQS gives managed scale, visibility timeouts, and DLQs, but you give up much of the custom Redis/Lua scheduling logic and local broker behavior.

PostgreSQL vs MongoDB

- PostgreSQL is better for this project because job state transitions, dedupe lookups, audit queries, indexes, and transactions are central.

- MongoDB would be fine for flexible payload storage, but less natural for advisory locks, relational DLQ references, and aggregate operational queries.

4. Scalability

Horizontal scaling model

- Scale API tasks behind a load balancer because enqueue uses shared Redis/PostgreSQL state.

- Scale workers by running more worker processes. Each worker uses the same queue name and gets unique workerId from env or process defaults.

- Increase per-process concurrency with WORKER_CONCURRENCY, but only until downstream services, Postgres pool, and job handler latency become bottlenecks.

- Scale dashboard read traffic separately from write path; SSE currently polls every two seconds per connected dashboard client.

Load	Likely first pressure point	Why	Mitigation
10x	PostgreSQL writes and connection pool	Every enqueue and state transition writes SQL; pool max is 20 per process.	Tune pool, add PgBouncer, batch writes where safe, monitor slow queries.
100x	Redis hot sorted set and Lua script	All workers contend on queue:<name>; claim script scans up to 50 ready jobs.	Shard queues, separate queues by workload, benchmark Lua latency, avoid huge ready bursts.
1000x	Single queue architecture and cross-store consistency	One Redis shard and one jobs table become coordination bottlenecks.	Partition by queue or tenant, Redis Cluster/MemoryDB, Postgres partitioning, outbox/reconciler, managed queue.

Bottlenecks to name

- Redis: single-threaded command execution per shard; Lua scripts block other Redis commands while they run.

- PostgreSQL: write throughput for enqueue, claim, ack/nack, heartbeat, audit, and DLQ; indexes improve reads but add write cost.

- Lock contention: advisory locks only affect duplicate idempotency keys, but the Redis queue key is shared by all workers for a queue.

- Worker runtime: Node handles I/O well, but CPU-bound handlers would block a worker process event loop.

- Dashboard metrics: COUNT queries over jobs can become expensive as the jobs table grows without retention or rollups.

Queue Sharding / Partitioning

What sharding would look like

- Redis key sharding: use queue:<queueName>:<partition> sorted sets and matching priority hashes. Producers choose a partition by hash of tenant, job id, or idempotency key.

- Worker assignment: each worker claims from one or more partitions. A simple version loops over partitions; a better version uses consistent hashing or explicit shard ownership.

- Postgres partitioning: partition jobs by queue_name, created_at, or hash of job id. Keep dead_letters and audit_log partitioned/retained too.

- Fairness: avoid one hot tenant starving others by using per-tenant queues or weighted fair scheduling.

- Ordering: this design does not promise strict FIFO globally. Sharding makes global ordering even less realistic, so communicate that clearly.

What breaks if you simply add workers forever

- More workers mean more claim calls, more lock operations, more Postgres UPDATE traffic, and more downstream pressure.

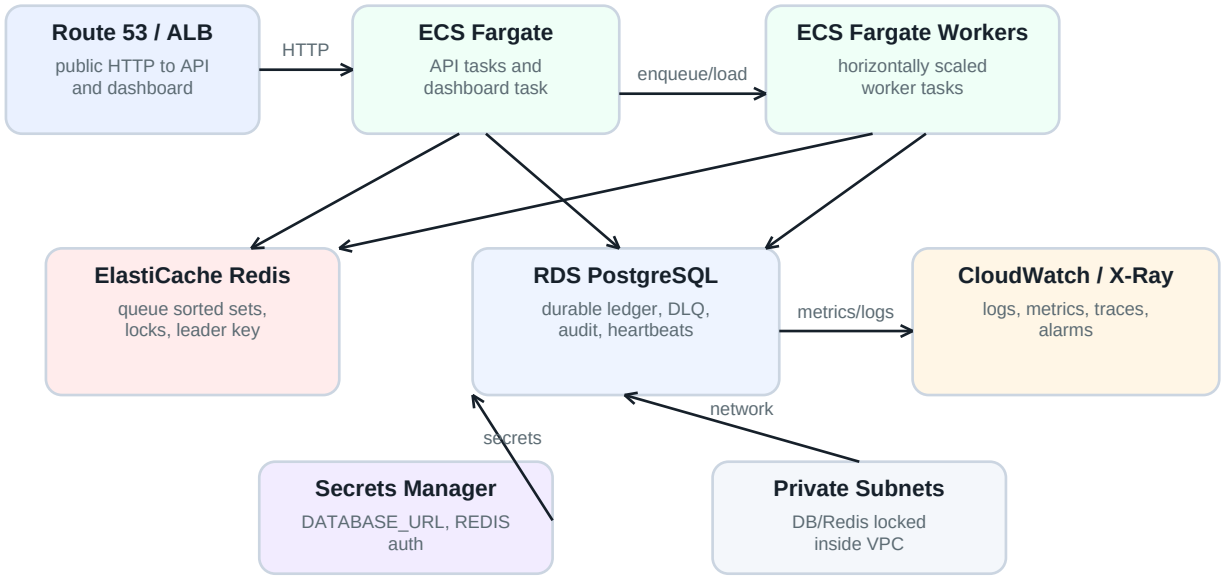
- Visibility timeout race gets more visible: slow jobs can be requeued while a worker still runs.

- Dead letter volume grows unless you add triage tooling, alerting, and retention.

- Backpressure is not currently enforced at the enqueue API. The API will accept work until Redis/Postgres or the process itself pushes back.

Source anchors: packages/core/src/lua/claim.lua:47-99; packages/core/src/db.ts:20-22; packages/dashboard/src/routes/metrics.ts:10-49; packages/worker/src/worker.ts:25-63.

5. AWS Deployment Mapping



Recommended AWS baseline

- Compute: ECS Fargate fits this worker model best for an interview baseline. API, dashboard, and workers are already containerized, and workers are stateless apart from Redis/Postgres.

- Use EKS if the team already standardizes on Kubernetes or needs advanced scheduling and custom controllers. Use EC2 only if you need host-level control or cheaper long-running capacity with more operational burden.

- Redis: ElastiCache Redis is the default managed mapping for current Redis usage. MemoryDB is stronger when Redis durability is a primary requirement. Self-managed Redis on EC2 is educational but operationally risky.

- Database: RDS PostgreSQL Multi-AZ is the direct durable store. Aurora PostgreSQL becomes attractive when write/read scale, faster failover, or storage autoscaling requirements outgrow standard RDS.

VPC, IAM, autoscaling, observability

- VPC: public ALB subnets; private subnets for ECS tasks; isolated/private subnets for RDS and Redis; security groups allow only required ports.

- IAM: task execution role for pulling images and writing logs; task role for Secrets Manager/Parameter Store access; least-privilege database and Redis credentials.

- Autoscaling: API scales on CPU/request count/latency; workers scale on queue depth, age of oldest job, jobs per worker, and DLQ/retry rate; dashboard stays small.

- Observability: CloudWatch logs and alarms at minimum; X-Ray/OpenTelemetry for request traces; Prometheus/Grafana equivalent if self-hosting metrics.

SQS + DLQ Alternative

If replaced with SQS + DLQ	Gain	Lose / rewrite
Queueing	Managed availability, throughput scaling, visibility timeout, redrive policy.	Custom Redis sorted set scheduling and Lua claim logic.
Retries	Native receive count and DLQ redrive can cover basic retry exhaustion.	Your exact backoff formula and priority sampling unless implemented outside SQS.
Priority	Could use multiple queues per priority tier.	Single-queue numeric priority behavior from Redis hash and Lua selection.
Idempotency	FIFO queues provide message dedup windows; Standard queues need app-level dedupe.	Your 24-hour PostgreSQL advisory-lock dedupe behavior unless retained.
Durable state	SQS stores messages, reducing custom broker ops.	PostgreSQL job ledger still needed if you want rich status queries, dashboard history, audit, replay semantics.
Ops	Less Redis management; AWS handles broker scaling.	Less opportunity to demonstrate low-level queue mechanics in this project.

Best interview framing

- I would use SQS for a production system if the business need is reliable managed queueing more than custom queue semantics.

- I built this Redis/Postgres design to demonstrate the mechanics SQS abstracts away: visibility timeouts, atomic claims, retry scheduling, dead letter replay, and idempotency.

- If migrating to SQS, I would likely keep PostgreSQL for job metadata, audit, dashboard, idempotency keys, and application-level replay records.

6. Production Readiness

What is missing right now

Area	Current state	Not yet implemented / suggested extension
Monitoring	Dashboard has queue depth, jobs/sec estimate, retry rate, DLQ size, worker liveness.	Prometheus/OpenTelemetry metrics, RED/USE dashboards, queue age, claim latency, handler duration histograms.
Alerting	No alert rules in repo.	DLQ growth, retry spike, no alive workers, old pending jobs, Redis/Pg connection failures, stale locks.
Backpressure	API validates required fields but no rate limit or queue-depth rejection.	Rate limiting, max queue depth, per-tenant quotas, 429/503 behavior.
Graceful shutdown	Worker stops claiming and waits up to 30 seconds for in-flight jobs.	Long-running job lock extension; SIGTERM tests; Kubernetes preStop/readiness drain.
Failover	ioredis reconnects; Pg pool logs errors. Compose has health checks.	Redis HA, RDS Multi-AZ, retry policies around failed writes, reconciler for Redis/Postgres divergence.
Security	Local hardcoded Compose credentials, CORS origin true, no API auth.	Secrets Manager, authN/authZ, TLS, private networking, input schema validation, least-privilege DB user.
Testing	Backoff unit tests pass; integration tests exist but are env-gated.	Always-on CI integration tests with ephemeral Redis/Pg, chaos tests, failover tests, race tests.

Source anchors: packages/worker/src/shutdown.ts:1-45; packages/core/src/redis.ts:17-24; docker-compose.yml:15-29; packages/dashboard/src/routes/metrics.ts:10-49; test result: 2 passed, 3 skipped on 2026-08-07.

Failover and Data Safety

Redis failover

- Current behavior: ioredis reconnects with a capped retry strategy. Redis locks are TTL-based, so some in-flight state can naturally expire.

- Risk: Redis is not the durable source of truth in this design, but the ready queue is in Redis. If Redis loses sorted set data, pending jobs can exist in PostgreSQL but not be claimable.

- Production answer: enable managed Redis HA/persistence, add a reconciler that periodically re-adds PostgreSQL pending jobs whose available_at <= now and are missing from Redis, and use an outbox pattern for enqueue.

PostgreSQL failover

- Current behavior: Pool errors are logged; SQL operations fail back to callers or worker loop error handling.

- Risk: a Redis claim can succeed while the PostgreSQL processing update fails, leaving the job removed from Redis until a reconciler fixes it.

- Production answer: RDS Multi-AZ/Aurora, conservative retry policy, idempotent SQL updates with locked_by checks, and a reconciliation job that treats PostgreSQL as source of truth.

Security posture

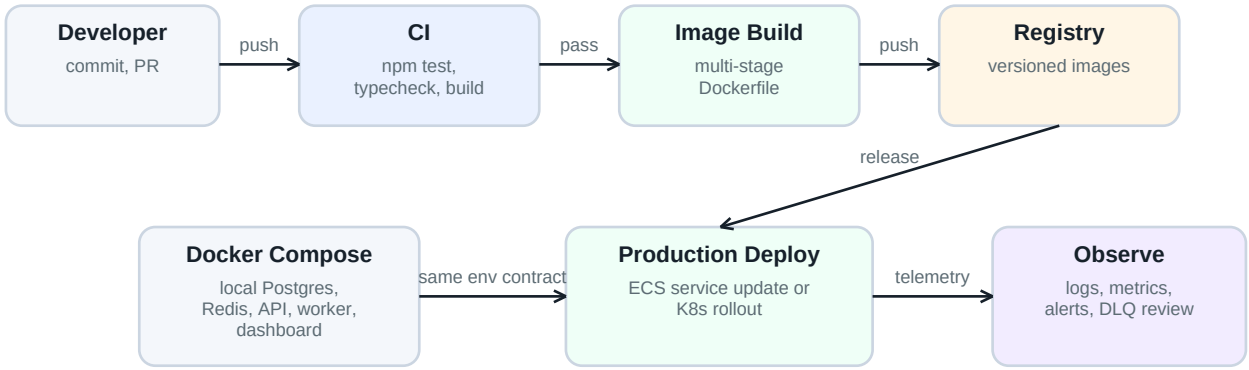
- Secrets: move DATABASE_URL and REDIS_URL credentials out of Compose literals into AWS Secrets Manager or Parameter Store.

- Network: expose only ALB/API/dashboard as needed; keep Redis and PostgreSQL private.

- Least privilege: separate DB users for migrations, API, workers, and dashboard if this became production.

- Input safety: add JSON schema validation for payload size, priority range, max_attempts, delay_ms, and idempotency key length.

7. Workflow / Operational Flow



Current repo note: no CI workflow file was found; CI/CD above is a suggested extension from existing npm and Docker scripts.

Current local flow

- Run full local stack with `docker compose up --build`. Compose defines Postgres, Redis, API, worker, and dashboard services.

- Postgres uses a named volume pgdata. Redis has no named persistence volume in Compose.

- The API and dashboard listen on 3000 and 3001. Redis is published locally as 6380 while containers use redis:6379.

- The Dockerfile is package-parameterized with PACKAGE_NAME and compiles core plus the selected package into a runtime image.

Source anchors: README.md Run With Docker; docker-compose.yml:5-90; Dockerfile:25-59; package.json:9-18.

Docker Compose to production path

- Step 1: keep the same env contract: DATABASE_URL, REDIS_URL, HOST, PORT, QUEUE_NAME, WORKER_CONCURRENCY, VISIBILITY_TIMEOUT_MS.

- Step 2: build and push separate API, worker, and dashboard images from the same Dockerfile with different PACKAGE_NAME args.

- Step 3: deploy API and dashboard behind ALB; deploy workers as private services with no public ingress.

- Step 4: replace Compose Postgres/Redis with RDS and ElastiCache/MemoryDB; run migrations through a controlled release job.

- Step 5: add autoscaling, alerts, dashboard auth, secret injection, and rollback strategy.

8. Specific Gotcha Questions

Question	Concise honest answer
Is this exactly-once?	No. It is an at-least-once queue. Redis locks and Lua reduce duplicate claims, but crashes and visibility expiry can re-deliver work. Handlers must be idempotent.
What happens if a job runs longer than 30 seconds?	Default visibilityTimeoutMs is 30000. There is no lock extension, so the reaper may requeue the job while the original worker still runs.
Can Redis and PostgreSQL get out of sync?	Yes. Enqueue, claim, ack, and nack coordinate two stores without a distributed transaction. A production version needs outbox/reconciliation.
What if Redis loses its data?	PostgreSQL still has job records, but pending jobs may be missing from Redis and not claimable until a reconciler rehydrates the sorted sets. That reconciler is not implemented.
Why Lua instead of MULTI/EXEC?	The script keeps candidate lookup, priority choice, lock acquire, and removal together in one server-side atomic operation with no client round trips between steps.
Does priority guarantee strict global priority?	No. Lua samples up to 50 ready jobs and chooses the highest priority within that batch. It is priority-biased, not a strict global priority scheduler.
How does the idempotency window work?	For same queue_name and idempotency_key, the code checks jobs created in the last 24 hours. Advisory lock serializes concurrent same-key requests.
Does idempotency dedupe completed side effects?	No. It only dedupes enqueue requests. The worker handler must make external side effects idempotent.
What happens after max attempts?	nack moves the job to status dead, inserts dead_letters, writes audit_log, clears locks, and removes Redis metadata.
Is the DLQ failure history complete?	Not fully. moveToDLQ currently creates a failureHistory array with the final failure; a history query exists but is not used to build multi-attempt history.

Gotcha Questions Continued

Question	Concise honest answer
Why is status failed in the schema?	The schema allows failed, but current nack logic uses pending for retry and dead for exhausted jobs. failed is not actively used.
Can any worker ack a job?	QueueService.ack updates by job id only and does not check locked_by or lock ownership. Production should verify ownership.
What if the worker crashes after Redis claim but before PostgreSQL update?	The job can be removed from Redis while still pending in PostgreSQL. The current reaper only checks processing jobs, so this needs reconciliation.
Can the reaper race with a slow worker ack?	Yes. Reaper UPDATE has a status guard, but the code does not check affected rows before re-adding to Redis. This is a limitation to fix.
How does graceful shutdown work?	The worker sets a draining flag, stops new claims, waits up to 30 seconds for active jobs, stops heartbeat/reaper, closes Redis and Postgres.
What is the bottleneck at high load?	Likely PostgreSQL write throughput and Redis hot queue key contention, then dashboard aggregate COUNT queries as history grows.
How would you scale workers?	Run more worker tasks and/or increase WORKER_CONCURRENCY. Then add queue sharding and autoscale on queue depth and oldest job age.
Why not use SQS?	For production simplicity I might. This project intentionally implements the queue mechanics itself to show atomic claims, backoff, DLQ replay, and idempotency.
Are integration tests always run?	No. Redis/Postgres tests are skipped unless RUN_INTEGRATION_TESTS=1. The default npm test run passed only the non-integration backoff tests today.
Is Compose production-ready?	No. It is a local demo/development stack. Production needs managed Redis/Postgres, secrets, private networking, autoscaling, alerts, and CI/CD.

Source anchors: Key limitations grounded in packages/core/src/queue.ts:112-137,145-169,228-282,351-386; packages/core/src/lua/claim.lua:47-99; packages/worker/src/shutdown.ts:1-45; packages/core/src/schema.sql:12.

One-Hour Talk Track

Minutes	Topic	What to emphasize
0-5	Problem and goal	Background work, retryable failures, durable state, operator recovery.
5-12	Architecture	API, Redis, PostgreSQL, workers, dashboard, reaper; draw data flow.
12-22	Job lifecycle	Enqueue, idempotency, Redis scheduling, Lua claim, ack/nack, backoff, DLQ.
22-32	Reliability mechanics	Visibility timeout, leader-elected reaper, at-least-once semantics, graceful shutdown.
32-40	Data model	jobs, dead_letters, worker_heartbeats, audit_log, indexes, why SQL.
40-48	Scale	Workers horizontally, bottlenecks, Redis hot key, Pg writes, partitioning strategy.
48-55	AWS	ECS Fargate, ALB, ElastiCache/MemoryDB, RDS/Aurora, CloudWatch, IAM, VPC.
55-60	Limitations and next steps	Cross-store consistency, reconciler/outbox, lock ownership, long job heartbeat, CI integration tests.

A strong closing sentence

This project is not trying to outdo SQS or RabbitMQ. It is a hands-on implementation of the reliability primitives those systems expose: atomic claiming, visibility timeouts, retry backoff, durable job state, and dead letter recovery. I can explain where it works, where it is intentionally simplified, and how I would harden it for production.

Cheat Sheet

Core answer

- System: Fastify API + Redis hot queue + PostgreSQL durable state + worker processes + dashboard + leader-elected reaper.

- Guarantee: at-least-once processing. Idempotency key dedupes enqueue for 24 hours, not job side effects.

- Redis keys: queue:<name> sorted set; queue:<name>:priorities hash; queue:lock:<jobId> TTL lock; reliable-queue:reaper:leader.

- Backoff: min(base * 2^attempt, cap) + random jitter up to 25 percent. Defaults base 1s, cap 60s.

- Visibility timeout: default 30s; lock TTL plus jobs.locked_until. Reaper scans expired processing jobs.

- DLQ: exhausted jobs get status dead, dead_letters row, audit_log row, and can be replayed.

Defend the stack

- Redis over RabbitMQ/SQS for custom mechanics and Lua atomicity in a learning/custom queue.

- PostgreSQL over MongoDB for transactions, advisory locks, indexes, audits, and operational SQL.

- TypeScript/Node for typed shared packages and I/O-heavy worker/API code.

- Docker Compose for repeatable local infra; ECS Fargate + RDS + ElastiCache is the clean AWS mapping.

Do not overclaim

- No exactly-once processing.

- No lock extension for long jobs.

- No cross-store transaction between Redis and PostgreSQL.

- No reconciler for pending jobs missing from Redis.

- No production auth, rate limiting, secrets manager, CI workflow, or full monitoring/alerting in repo.

- Integration tests exist but were skipped in the default test run unless RUN_INTEGRATION_TESTS=1.

Best next improvements

- Add outbox/reconciler to repair Redis from PostgreSQL.

- Add lock ownership checks to ack/nack and reaper rowCount checks.

- Add visibility lock extension heartbeat for long-running jobs.

- Add schema validation, auth, rate limiting, OpenTelemetry metrics, alerting, and always-on integration tests.

Source anchors: Cheat sheet sources: packages/core/src/config.ts:61-68; packages/core/src/redis.ts:52-67; packages/core/src/backoff.ts:20-31; packages/core/src/queue.ts:28-386; packages/worker/src/reaper.ts:79-172.